

Python pour le trading — Document 1/8

Les fondamentaux du langage

1. Les types de données

Avant tout calcul, il faut décider où ranger l'information. Python offre quatre structures de base. Choisir la bonne est la première marque d'un développeur compétent. Pour bien les distinguer, on applique à chacune les trois mêmes opérations : ajouter un élément, modifier un élément existant, tester si un élément est présent. Les différences de comportement sautent alors aux yeux.

1.1 La liste : une séquence ordonnée et modifiable

Une liste retient des éléments dans l'ordre où on les insère, et on peut la changer librement. C'est le choix par défaut pour une suite qui évolue dans le temps, comme les ordres reçus au fil de la journée.

```
ordres = ["ES", "NQ", "BTC"]
ordres.append("CL")          # AJOUTER -> ["ES", "NQ", "BTC", "CL"]
ordres[0] = "GC"             # MODIFIER -> ["GC", "NQ", "BTC", "CL"]
"NQ" in ordres                # TESTER -> True
```

La liste accepte les trois opérations. Son point faible est le test de présence (`in`) : Python parcourt les éléments un par un jusqu'à trouver la valeur. Instantané sur 10 éléments, ce parcours devient coûteux sur plusieurs millions. On dit que ce test est en **O(n)** : le temps augmente proportionnellement au nombre d'éléments.

1.2 Le tuple : une séquence ordonnée mais figée

Le tuple ressemble à la liste, à une différence majeure : une fois créé, il ne peut plus être modifié. Cette rigidité est un avantage quand la donnée représente un fait définitif, comme une transaction déjà exécutée qui ne doit jamais changer par accident.

```
ordres = ("ES", "NQ", "BTC")
ordres.append("CL")          # AJOUTER -> AttributeError (impossible)
ordres[0] = "GC"             # MODIFIER -> TypeError (impossible)
"NQ" in ordres                # TESTER -> True
```

Les deux premières opérations échouent volontairement : Python lève une erreur dès qu'on tente de modifier un tuple. Cette immuabilité protège les données critiques. Le test de présence fonctionne mais reste en **O(n)**, car le tuple est lui aussi parcouru élément par élément.

1.3 Le dictionnaire : des paires clé-valeur à accès rapide

Le dictionnaire associe une clé à une valeur, comme un répertoire associe un nom à un numéro. C'est l'outil idéal pour décrire un objet par ses attributs : un ordre a un symbole, une quantité, un prix. On accède à chaque information par son nom de clé, et cet accès est quasi instantané quelle que soit la taille du dictionnaire.

```
ordre = {"symbole": "ES", "quantite": 5}
ordre["prix"] = 5234.50      # AJOUTER -> ajoute la clé "prix"
ordre["quantite"] = 3        # MODIFIER -> change la valeur existante
"prix" in ordre               # TESTER -> True (rapide)
```

Ajouter et modifier se font par la clé : si la clé existe, sa valeur est remplacée ; sinon, la paire est créée. Le grand atout est le test de présence en **O(1)** : le dictionnaire utilise une table de hachage qui trouve la clé directement, sans parcourir tout le contenu. Le temps de recherche ne dépend pas du nombre d'éléments. À noter : le test `in` porte sur les clés, pas sur les valeurs.

1.4 Le set : un ensemble sans doublons ni ordre

Le set est une collection sans doublon et sans ordre. On l'utilise quand seule compte la présence ou l'absence d'une valeur, par exemple la liste des symboles distincts tradés aujourd'hui, peu importe combien de fois chacun apparaît.

```
symboles = {"ES", "NQ", "BTC"}
symboles.add("CL")      # AJOUTER -> {"ES", "NQ", "BTC", "CL"}
symboles.add("NQ")      # doublon ignoré -> ensemble inchangé
"NQ" in symboles        # TESTER -> True (rapide)
```

On ajoute avec `add`, mais il n'existe pas de modification par position : un set n'a pas d'index, puisqu'il n'est pas ordonné. Une valeur déjà présente est simplement ignorée. Comme le dictionnaire, le set teste la présence en **O(1)** grâce au hachage.

1.5 Tableau de synthèse

	ordonné	modifiable	doublons	test "in"
Liste	oui	oui	oui	O(n) lent
Tuple	oui	NON	oui	O(n) lent
Dictionnaire	oui*	oui	clés: non	O(1) rapide
Set	non	oui	NON	O(1) rapide

(* le dict conserve l'ordre d'insertion depuis Python 3.7)

Pourquoi ça compte. Le choix de structure a un effet direct sur la vitesse. Si un système vérifie des milliers de fois par seconde « ai-je déjà une position sur ce symbole ? », le faire sur un set ou un dict ($O(1)$) plutôt qu'une liste ($O(n)$) peut faire la différence entre un système réactif et un système qui prend du retard sur le marché.

2. Le slicing (découpage de séquences)

Le slicing extrait une sous-partie d'une séquence sans écrire de boucle. La syntaxe est `[début:fin:pas]`, où la borne de fin est toujours exclue. Mais une condition est essentielle : le slicing exige un ordre et un index numérique (les positions 0, 1, 2...). On va donc l'appliquer aux quatre structures pour voir lesquelles le permettent — et c'est très instructif.

2.1 La liste : le slicing fonctionne

La liste est ordonnée et indexée par position : le slicing y fonctionne parfaitement. C'est le cas d'usage principal.

```
prix = [5230.0, 5231.5, 5232.0, 5233.25, 5234.50, 5235.0]

prix[-3:]    # 3 DERNIERS    -> [5233.25, 5234.50, 5235.0]
prix[:2]     # 2 PREMIERS    -> [5230.0, 5231.5]
```

```
prix[1:4]      # INDICES 1 à 3 -> [5231.5, 5232.0, 5233.25]
prix[:: -1]    # INVERSÉ      -> [5235.0, 5234.50, ... , 5230.0]
prix[::2]      # UN SUR DEUX  -> [5230.0, 5232.0, 5234.50]
```

Un indice négatif compte depuis la fin : -1 est le dernier élément, donc `prix[-3:]` signifie « à partir du troisième en partant de la fin, jusqu'au bout ». Omettre une borne signifie « depuis le début » ou « jusqu'à la fin ». Le troisième nombre est le pas : un pas négatif parcourt la séquence à l'envers, ce qui explique l'inversion avec `[::-1]`.

2.2 Le tuple : le slicing fonctionne aussi

Le tuple est lui aussi ordonné et indexé : le slicing y fonctionne exactement comme sur une liste. La seule différence est que le résultat est un tuple, et non une liste — logique, puisqu'on découpe un tuple.

```
prix = (5230.0, 5231.5, 5232.0, 5233.25, 5234.50, 5235.0)

prix[-3:]      # 3 DERNIERS -> (5233.25, 5234.50, 5235.0)    <- un tuple
prix[1:4]      # PLAGE      -> (5231.5, 5232.0, 5233.25)
prix[:: -1]    # INVERSÉ    -> (5235.0, ... , 5230.0)
```

On retrouve les mêmes règles. Le découpage d'un tuple renvoie un nouveau tuple ; l'original, lui, reste intact (rappel : un tuple est immuable, mais le slicing ne le modifie pas, il en crée une copie partielle).

2.3 Le dictionnaire : le slicing ne fonctionne PAS

Le dictionnaire ne se découpe pas. Il est accédé par clé, pas par position numérique : la notion de « du 1er au 3e » n'a pas de sens pour lui. Tenter un slicing lève une erreur.

```
ordre = {"symbole": "ES", "quantite": 5, "prix": 5234.50}

ordre[0:2]      # -> TypeError : unhashable type: 'slice'
                #      (Python croit que 0:2 est une CLÉ, qui n'existe pas)
```

L'explication est subtile : sur un dict, les crochets servent à chercher une **clé**, pas une position. Python interprète donc `0:2` comme une tentative de clé de type « tranche », ce qui est invalide. Si l'on veut malgré tout les « N premières » paires, il faut d'abord transformer le dict en quelque chose d'ordonné et d'indexable :

```
# Solution : passer par une liste des paires (items), qui, elle, se découpe
list(ordre.items())[0:2]    # -> [("symbole", "ES"), ("quantite", 5)]
```

2.4 Le set : le slicing ne fonctionne PAS non plus

Le set est encore plus catégorique : il n'a aucun ordre ni index. Il n'y a donc ni « premier » ni « troisième » élément. Le slicing est impossible, et même un simple accès par position l'est.

```
symboles = {"ES", "NQ", "BTC"}

symboles[0]      # -> TypeError : 'set' object is not subscriptable
symboles[0:2]    # -> TypeError : même raison
```

« not subscriptable » signifie « on ne peut pas utiliser les crochets [] dessus ». Comme pour le dict, si l'on a besoin d'un découpage, on convertit d'abord le set en liste — mais attention, l'ordre obtenu n'est pas garanti, justement parce qu'un set n'a pas d'ordre :

```
list(symboles)[:2] # fonctionne, mais l'ordre des 2 éléments est imprévisible
```

Pourquoi ça compte. Le slicing est réservé aux séquences ordonnées et indexées : liste, tuple, et aussi les chaînes de caractères. Le dict s'accède par clé et le set n'a pas d'ordre, donc aucun des deux ne se découpe. Le `retenir` évite une erreur très fréquente chez les débutants. « Donne-moi les N dernières barres » avec `prix[-N:]` est l'opération la plus courante en analyse de marché, et elle suppose justement une liste ou un tuple.

3. Les compréhensions

D'abord, qu'est-ce qu'une « collection » ? C'est simplement un terme générique pour désigner une structure qui contient plusieurs éléments : une liste, un tuple, un dictionnaire et un set sont tous des collections. Quand on dit qu'une compréhension « construit une collection », cela veut dire qu'elle fabrique l'une de ces structures — par exemple une nouvelle liste ou un nouveau dictionnaire.

Une compréhension construit donc une nouvelle collection à partir d'une collection existante, en une seule ligne lisible, là où l'on écrirait sinon une boucle avec ajout répété. C'est l'une des constructions les plus caractéristiques de Python. Selon les délimiteurs employés, le résultat est une liste, un dictionnaire, un set ou un générateur.

Pour bien voir l'intérêt, comparons d'abord avec et sans compréhension. Les deux blocs suivants produisent le même résultat — la liste des prix — mais le second est plus court et plus lisible.

```
ordres = [
    {"symbole": "ES", "cote": "achat", "quantite": 5, "prix": 5234.50},
    {"symbole": "NQ", "cote": "vente", "quantite": 2, "prix": 18450.0},
    {"symbole": "ES", "cote": "achat", "quantite": 3, "prix": 5235.00},
]
```

```
# SANS compréhension : une boucle classique
prix = []
for o in ordres:
    prix.append(o["prix"])
# prix vaut [5234.50, 18450.0, 5235.00]
```

```
# AVEC compréhension : une seule ligne, même résultat
prix = [o["prix"] for o in ordres]
```

La structure d'une compréhension est toujours la même : une **expression** (ce qu'on veut produire), suivie de `for ... in ...` (la source qu'on parcourt), et éventuellement une condition `if` (un filtre). À partir de cette même base, on change la collection produite selon les délimiteurs. Décortiquons maintenant chaque forme, une par une.

Au sujet du nom de variable. Dans `[ordre["prix"] for ordre in ordres]`, le mot `ordre` est un nom de variable temporaire, totalement arbitraire : c'est nous qui le choisissons. Il représente « l'élément courant » pendant le parcours. On pourrait écrire `x`, `o` ou `element` à la place, tant qu'on emploie le même nom des deux côtés du `for`. Ici on choisit `ordre` parce que c'est parlant : on parcourt des ordres. Le nom n'a aucun effet sur le résultat, il sert seulement à la lisibilité.

3.1 Compréhension de liste : extraire une valeur

```
[ordre["prix"] for ordre in ordres]    # -> [5234.50, 18450.0, 5235.00]
```

Lecture de gauche à droite : `for ordre in ordres` parcourt la liste, en nommant « ordre » chaque élément tour à tour ; `ordre["prix"]` est l'expression appliquée à chacun, soit « le prix de cet ordre ». Le résultat est une nouvelle liste contenant le prix de chaque ordre. En clair : « pour chaque ordre dans ordres, prends son prix ».

3.2 Compréhension de liste avec filtre

```
[ordre for ordre in ordres if ordre["cote"] == "achat"]    # -> 2 ordres
```

Même principe, plus un filtre à la fin. `for ordre in ordres` parcourt les ordres ; `if ordre["cote"] == "achat"` ne garde que ceux dont le côté vaut « achat » ; l'expression de tête, ici simplement `ordre`, indique qu'on garde l'ordre entier (et non un de ses champs). En clair : « pour chaque ordre, garde-le seulement si c'est un achat ». Sur nos trois ordres, deux passent le filtre.

3.3 Compréhension de dictionnaire

```
{ordre["symbole"]: ordre["quantite"] for ordre in ordres}    # -> {"ES": 3, "NQ": 2}
```

Ici l'expression de tête a deux parties séparées par un deux-points : `ordre["symbole"]` est la **clé** et `ordre["quantite"]` est la **valeur**. Pour chaque ordre, on crée une paire « symbole : quantité ». Comme un dictionnaire ne peut pas avoir deux fois la même clé, le second ordre ES (quantité 3) écrase le premier (quantité 5) : d'où `ES = 3`. C'est le deux-points dans l'expression qui fait que Python produit un dict plutôt qu'un set.

Avec filtre. On peut aussi ajouter une condition, exactement comme pour une liste — par exemple ne garder que les ordres d'achat :

```
{ordre["symbole"]: ordre["prix"] for ordre in ordres if ordre["cote"] == "achat"}  
# -> {"ES": 5235.00}    (seuls les achats ; le dernier ES écrase le premier)
```

3.4 Compréhension de set

```
{ordre["symbole"] for ordre in ordres}    # -> {"ES", "NQ"}
```

Mêmes accolades que le dictionnaire, mais **sans** deux-points : Python produit alors un set. `ordre["symbole"]` extrait le symbole de chaque ordre ; comme un set élimine les doublons, les deux ordres ES ne donnent qu'un seul « ES ». Le résultat est l'ensemble des symboles distincts présents dans les ordres.

Avec filtre. Là encore, on peut filtrer — par exemple les symboles distincts des seuls ordres de vente :

```
{ordre["symbole"] for ordre in ordres if ordre["cote"] == "vente"}  
# -> {"NQ"}    (un seul ordre de vente dans notre exemple)
```

3.5 Expression génératrice

```
sum(ordre["quantite"] * ordre["prix"] for ordre in ordres)    # somme des valeurs
```

Avec des parenthèses, on obtient un générateur. `ordre["quantite"] * ordre["prix"]` calcule, pour chaque ordre, sa valeur notionnelle (quantité fois prix). Ici, le générateur est passé directement à la fonction `sum`, qui additionne ces valeurs une par une à mesure qu'elles sont produites. La différence essentielle avec une liste : le générateur ne construit jamais la liste complète en mémoire ; il fournit

chaque valeur à la demande, puis l'oublie. Sur trois ordres c'est sans effet, mais sur des millions de lignes, cela évite de saturer la mémoire.

Pour résumer, ce sont les délimiteurs qui décident de la nature du résultat : crochets pour une liste, accolades avec deux-points pour un dictionnaire, accolades simples pour un set, parenthèses pour un générateur. Et dans les quatre cas, on peut ajouter un filtre `if` à la fin.

3.6 Variations utiles : `if/else`, `zip`, imbrication

Trois variations reviennent souvent et méritent d'être connues.

Un **`if/else`** dans l'expression. À ne pas confondre avec le filtre : le filtre `if` en fin de ligne **garde ou jette** un élément ; un `if/else` placé dans l'expression de tête **transforme** chaque élément, sans jamais en jeter.

```
# Étiqueter chaque ordre selon son côté (if/else DANS l'expression)
["long" if o["cote"] == "achat" else "short" for o in ordres]
# -> ['long', 'short', 'long']
```

Ici, on produit toujours trois éléments (un par ordre), mais chacun est remplacé par « long » ou « short » selon son côté. Le `if/else` est avant le `for` (il fait partie de l'expression), alors qu'un filtre se placerait après.

Parcourir deux listes de front avec **`zip`**. `zip` associe les éléments de même position de plusieurs séquences.

```
# Construire un dict en associant deux listes parallèles
symboles = ["ES", "NQ", "CL"]
prix      = [5234.50, 18450.0, 78.90]
{s: p for s, p in zip(symboles, prix)}
# -> {"ES": 5234.50, "NQ": 18450.0, "CL": 78.90}
```

`zip(symboles, prix)` fournit les paires ("ES", 5234.50), ("NQ", 18450.0)... que la compréhension transforme en entrées du dictionnaire. C'est le moyen idiomatique de recombinaison des colonnes parallèles.

Une compréhension **imbriquée**. On peut enchaîner deux `for`, lus de gauche à droite : le premier est la boucle extérieure, le second la boucle intérieure. Cas classique : aplatir une liste de listes.

```
# Aplatir des carnets (liste de listes) en une seule liste de prix
carnets = [[5234.50, 5234.75], [18450.0, 18451.0]]
[prix for carnet in carnets for prix in carnet]
# -> [5234.50, 5234.75, 18450.0, 18451.0]
```

On lit l'imbrication comme deux boucles emboîtées : « pour chaque carnet dans carnets, puis pour chaque prix dans ce carnet, garde ce prix ». Le résultat est une liste plate réunissant tous les prix.

Pourquoi ça compte. Au-delà de la concision, une compréhension est généralement plus rapide qu'une boucle équivalente. Et l'expression génératrice permet de traiter des volumes de données de marché qui ne tiendraient pas en mémoire d'un seul bloc.

4. Les chaînes de caractères et les f-strings

Le texte est partout : messages reçus des plateformes, journaux d'activité, rapports. Savoir le formater et le manipuler est essentiel. Les f-strings (chaînes préfixées de la lettre f) sont la façon moderne d'insérer des variables dans du texte : il suffit de placer la variable entre accolades.

```
symbole, quantite, prix = "ES", 5, 5234.50

f"Ordre : achat {quantite} {symbole} @ {prix}" # "Ordre : achat 5 ES @ 5234.5"
f"{quantite * prix:,.2f} $"                  # "26,172.50 $"
f"{symbole:<6}{prix:>10.2f}"                  # "ES          5234.50"
```

On peut mettre en forme directement dans les accolades, après un deux-points. `,.2f` affiche un nombre avec séparateur de milliers et deux décimales, indispensable pour des montants lisibles. `<6` et `>10` alignent le texte à gauche ou à droite sur une largeur donnée, pour obtenir des colonnes bien alignées dans un journal.

Au-delà du formatage, quelques méthodes reviennent constamment lorsqu'on reçoit des données textuelles brutes :

```
" es ".strip().upper()          # "ES"                (nettoyer une saisie)
"ES,NQ,BTC".split(",")         # ["ES", "NQ", "BTC"] (découper un message)
"-".join(["ES", "achat", "5"]) # "ES-achat-5"      (recomposer une clé)
```

`strip` retire les espaces superflus en début et fin ; `split` découpe une chaîne en liste selon un séparateur ; `join` fait l'inverse en assemblant une liste en une seule chaîne. Ces méthodes s'enchaînent de gauche à droite, chacune opérant sur le résultat de la précédente.

Pourquoi ça compte. Les données d'une API arrivent souvent sous forme de texte, par exemple « ES,achat,5,5234.50 ». Les nettoyer, les découper en champs puis les recomposer est une tâche de tous les instants dans un système connecté aux marchés.

5. Les fonctions

Une fonction regroupe une logique sous un nom, pour la réutiliser sans la réécrire. C'est la première brique d'un code organisé. Les valeurs qu'on lui passe sont les arguments, et Python offre plusieurs manières de les fournir.

```
def creer_ordre(symbole, quantite, prix, cote="achat"):
    return {"symbole": symbole, "quantite": quantite,
            "prix": prix, "cote": cote}

creer_ordre("ES", 5, 5234.50)          # appel POSITIONNEL
creer_ordre(symbole="NQ", prix=18450.0, # appel NOMMÉ
            quantite=2, cote="vente")
```

L'argument `cote` a une valeur par défaut : si on ne le précise pas, il vaut « achat ». Dans l'appel positionnel, c'est l'ordre des valeurs qui détermine à quel paramètre chacune correspond. Dans l'appel nommé, on indique explicitement le nom de chaque paramètre, ce qui rend le code plus lisible et libère de l'ordre des arguments.

5.1 Un nombre variable d'arguments : `*args` et `**kwargs`

Normalement, une fonction déclare un nombre fixe d'arguments. Mais parfois on veut en accepter un nombre indéterminé. Python offre deux mécanismes complémentaires, et la différence entre les deux tient à **un seul détail : l'argument est-il nommé ou non ?**

***args** (un seul astérisque) rassemble tous les arguments **positionnels** en trop — ceux passés sans nom, juste une valeur — dans un **tuple**.

****kwargs** (deux astérisques) rassemble tous les arguments **nommés** en trop — ceux passés sous la forme nom=valeur — dans un **dictionnaire**. (« kwargs » est l'abréviation de *keyword arguments*, arguments par mot-clé.)

Le plus parlant est de voir, sur un même appel, ce qui atterrit dans chacun :

```
def inspecter(*args, **kwargs):
    print("args  =", args)      # un TUPLE
    print("kwargs =", kwargs)   # un DICT

inspecter("ES", 5, 5234.50, cote="achat", source="bot")
```

Affiche :

```
args  = ('ES', 5, 5234.50)
kwargs = {'cote': 'achat', 'source': 'bot'}
```

On voit nettement la répartition. Les trois premières valeurs ("ES", 5, 5234.50) sont passées sans nom : ce sont des arguments positionnels, donc elles vont dans args sous forme de tuple. Les deux suivantes (cote="achat" et source="bot") sont passées avec un nom : ce sont des arguments nommés, donc elles vont dans kwargs sous forme de dictionnaire, où le nom devient la clé et la valeur la valeur.

À retenir : ce n'est pas la nature de la donnée qui décide, mais **la façon dont on la passe**. La même valeur 5 irait dans args si on écrit inspecter(5), et dans kwargs si on écrit inspecter(quantite=5). Les étoiles, elles, servent à « emballer » ces arguments à la définition de la fonction ; sans elles, la fonction n'accepterait qu'un nombre fixe d'arguments.

Ces deux types sont fixes. *args est **toujours** un tuple, et **kwargs est **toujours** un dictionnaire — jamais une liste ni un set. Ce n'est pas un choix de notre part, c'est ainsi que Python est conçu. Le tuple convient pour args car les arguments positionnels ont un ordre (le 1er, le 2e...) et n'ont pas à être modifiés ; le dict convient pour kwargs car chaque argument nommé est naturellement une paire nom-valeur. Les noms « args » et « kwargs » sont eux aussi une simple convention : seules les étoiles comptent pour Python, mais tout le monde utilise ces deux noms par habitude.

5.2 Un piège classique : l'argument mutable par défaut

Voici une erreur subtile qui revient souvent en entretien, parce qu'elle révèle si l'on comprend à quel moment les valeurs par défaut sont créées.

```
# MAUVAIS
def ajouter(ordre, panier=[]):
    panier.append(ordre)
    return panier

ajouter("ES")    # ["ES"]
ajouter("NQ")    # ["ES", "NQ"]  <- l'ES précédent est resté !
```


Le problème : la liste par défaut est créée une seule fois, au moment où Python lit la définition de la fonction, et non à chaque appel. Tous les appels partagent donc la même liste, et les éléments s'y accumulent. On corrige en utilisant None comme valeur par défaut, puis en créant une liste neuve à l'intérieur de la fonction :

```
# BON
def ajouter(ordre, panier=None):
    if panier is None:
        panier = []
    panier.append(ordre)
    return panier
```

Pourquoi ça compte. Dans un système de trading, ce bug ferait s'accumuler des ordres fantômes entre des appels censés être indépendants, ce qui pourrait provoquer des opérations involontaires et coûteuses.

6. La gestion des exceptions

Une exception est une erreur qui, si elle n'est pas interceptée, interrompt brutalement le programme. Dans un système connecté aux marchés en continu, ce serait catastrophique : une seule donnée corrompue ou une coupure d'API ne doit jamais arrêter tout le système. On encadre donc le code risqué pour attraper les erreurs et y réagir proprement.

```
def executer(ordre, solde):
    try:
        cout = ordre["quantite"] * ordre["prix"]
        if cout > solde:
            raise ValueError("Solde insuffisant")
        return {"statut": "execute"}

    except KeyError as e:
        # ordre mal formé : un champ manque
        return {"statut": "rejete", "raison": f"champ {e} manquant"}

    except ValueError as e:
        # erreur métier prévue
        return {"statut": "rejete", "raison": str(e)}

    except Exception as e:
        # filet de sécurité pour l'imprévu
        return {"statut": "erreur", "raison": str(e)}

    finally:
        # s'exécute TOUJOURS
        print("Tentative terminée.")
```

Le bloc try contient le code susceptible d'échouer. Chaque except attrape un type d'erreur précis : un KeyError indique qu'un champ attendu est absent (ordre mal formé), un ValueError signale ici une règle métier non respectée. L'ordre des except compte : on va du plus spécifique au plus général. Le dernier, except Exception, sert de filet de sécurité pour toute erreur imprévue.

Le bloc finally s'exécute dans tous les cas, succès comme échec. C'est l'endroit où l'on libère les ressources : fermer une connexion, relâcher un verrou, écrire dans un journal. On a ainsi la garantie que ce nettoyage a lieu même si une erreur survient.

6.1 Les types d'erreurs courants

Python fournit de nombreux types d'exceptions intégrés. En attraper un précis permet de réagir de façon ciblée. Voici les plus fréquents, avec un exemple de ce qui les déclenche :

KeyError	une clé absente d'un dict	ordre["prix"] si "prix" n'existe pas
IndexError	un index hors limites	prix[99] sur une liste de 6 éléments
ValueError	une valeur du bon type mais négative	int("abc") / quantité
TypeError	invalid une opération entre types incompatibles	"5" + 3 (texte + nombre)
ZeroDivisionError	une division par zéro	pnl / nb_trades si nb_trades = 0
AttributeError	une méthode/attribut inexistant	un_tuple.append(x)
FileNotFoundError	un fichier introuvable	open("data_inexistante.csv")
KeyboardInterrupt	l'utilisateur arrête le programme	Ctrl+C

Tous ces types descendent d'une classe commune, `Exception`. C'est pourquoi `except Exception` les attrape tous : il sert de filet de sécurité général, à placer en dernier, après les `except` spécifiques.

6.2 Définir sa propre exception

Au-delà des erreurs intégrées, on peut créer ses propres types d'erreurs pour exprimer des règles métier. L'intérêt : un nom parlant, et la possibilité d'attraper précisément cette erreur-là sans la confondre avec les autres.

Créer une exception personnalisée tient en deux lignes : on définit une classe qui hérite de `Exception`.

```
class OrdreInvalideError(Exception):  
    pass
```

Le mot `pass` signifie « rien de plus à ajouter » : hériter de `Exception` suffit à obtenir une exception pleinement fonctionnelle. Voyons maintenant le cycle complet — la **lever** quand une règle est violée, puis l'**attraper** à l'endroit qui appelle.

Étape 1 — lever l'exception avec `raise`, lorsqu'un ordre ne respecte pas nos règles :

```
def valider(ordre):  
    if ordre["quantite"] <= 0:  
        raise OrdreInvalideError("La quantité doit être positive")  
    if ordre["prix"] <= 0:  
        raise OrdreInvalideError("Le prix doit être positif")  
    return True
```

Étape 2 — attraper l'exception là où on appelle `valider`, pour réagir proprement au lieu de planter :

```
mauvais_ordre = {"symbole": "ES", "quantite": -5, "prix": 5234.50}  
  
try:  
    valider(mauvais_ordre)  
    print("Ordre valide, on l'envoie au marché")  
except OrdreInvalideError as e:  
    print(f"Ordre rejeté : {e}")    # -> "Ordre rejeté : La quantité doit être positive"
```

Le déroulement : `valider` détecte la quantité négative et lève `OrdreInvalideError` avec un message. L'exécution saute aussitôt au bloc `except` correspondant, qui récupère l'exception dans la variable `e` et affiche son message. La ligne « on l'envoie au marché » n'est jamais atteinte, car la levée interrompt le `try`. L'intérêt d'un type dédié : on peut écrire `except OrdreInvalideError` pour ne traiter que ce cas précis, en laissant les autres erreurs (réseau, etc.) suivre leur propre chemin.

Pourquoi ça compte. La règle d'or du temps réel est de « fail gracefully » : attraper l'erreur sur un ordre, la journaliser, et continuer de traiter les suivants. Le système reste opérationnel au lieu de s'effondrer sur un seul incident.

Récapitulatif

- 1. Types de données** : la liste évolue, le tuple est figé, le dictionnaire et le set offrent un test de présence instantané. Le bon choix conditionne la performance.
- 2. Slicing** : découper liste et tuple fonctionne ; sur dict et set c'est impossible (pas d'index). `prix[-N:]` donne les N derniers éléments.
- 3. Compréhensions** : construire une collection (liste, dict, set, générateur) en une ligne ; le générateur économise la mémoire sur les gros flux.
- 4. f-strings** : insérer et mettre en forme des variables dans du texte ; manipuler les chaînes avec `strip`, `split` et `join`.
- 5. Fonctions** : arguments positionnels ou nommés, valeurs par défaut, `*args/**kwargs`, et le piège de l'argument mutable par défaut.
- 6. Exceptions** : attraper chaque type d'erreur séparément, garder un filet de sécurité, nettoyer dans `finally`. Fail gracefully.

Questions de vérification

Essaie de répondre avant de lire la réponse, fournie juste en dessous, en retrait.

Sur les types de données

Q1. Pourquoi utiliser un set plutôt qu'une liste pour tester l'appartenance (`in`) ?

Réponse. Le test est en $O(1)$ sur un set, contre $O(n)$ sur une liste. Le set utilise une table de hachage qui trouve directement la valeur, alors que la liste doit la chercher en parcourant ses éléments un par un. Sur de gros volumes ou des tests très fréquents, l'écart de vitesse est énorme.

Q2. Quelle structure choisirais-tu pour une transaction déjà exécutée, et pourquoi ?

Réponse. Un tuple, parce qu'une transaction exécutée est un fait définitif. L'immutabilité du tuple empêche toute modification accidentelle de cette donnée critique.

Q3. Que se passe-t-il si on ajoute deux fois le même symbole à un set ?

Réponse. Le doublon est ignoré. Un set ne contient jamais deux fois la même valeur, donc l'ensemble reste inchangé.

Q4. Sur quoi porte le test `in` dans un dictionnaire : les clés ou les valeurs ?

Réponse. Sur les clés. "prix" in ordre vérifie l'existence de la clé "prix", pas d'une valeur.

Q5. Que signifie « $O(1)$ » par opposition à « $O(n)$ » ?

Réponse. $O(1)$ signifie que le temps de l'opération ne dépend pas du nombre d'éléments : il reste constant. $O(n)$ signifie qu'il augmente proportionnellement au nombre d'éléments. Tester une présence est $O(1)$ sur un dict/set et $O(n)$ sur une liste/tuple.

Sur le slicing

Q6. Sur quelles structures le slicing fonctionne-t-il, et pourquoi pas sur les autres ?

Réponse. Il fonctionne sur la liste et le tuple (et les chaînes), car ils sont ordonnés et indexés par position. Il ne fonctionne ni sur le dict (accédé par clé, pas par position) ni sur le set (aucun ordre ni index).

Q7. Que renvoie `prix[-3:]` sur une liste, et pourquoi est-ce utile en analyse de marché ?

Réponse. Les trois derniers éléments. C'est la base d'une fenêtre glissante : « les N dernières barres », nécessaire pour une moyenne mobile ou l'évaluation d'un signal récent.

Q8. Quelle erreur obtient-on en faisant `mon_set[0]`, et que signifie-t-elle ?

Réponse. `TypeError : 'set' object is not subscriptable`. Cela signifie qu'on ne peut pas utiliser les crochets [] sur un set, car il n'a pas d'index.

Q9. Comment obtenir les 2 premières paires d'un dictionnaire malgré l'absence de slicing ?

Réponse. En convertissant d'abord les paires en liste, qui se découpe : `list(ordre.items())[:2]`.

Q10. Quelle est la différence entre `prix[1:4]` et `prix[1:4:2]` ?

Réponse. `prix[1:4]` prend les indices 1, 2 et 3 (la fin est exclue). `prix[1:4:2]` ajoute un pas de 2 et ne prend donc que les indices 1 puis 3.

Sur les compréhensions

Q11. Qu'appelle-t-on une « collection » en Python ?

Réponse. Un terme générique pour une structure contenant plusieurs éléments : liste, tuple, dictionnaire et set sont tous des collections.

Q12. Dans `[ordre["prix"] for ordre in ordres]`, le mot « ordre » est-il imposé par Python ?

Réponse. Non, c'est un nom de variable temporaire totalement arbitraire, qu'on choisit pour la lisibilité. On pourrait écrire `x` ou `o` à la place, du moment qu'on emploie le même nom des deux côtés du `for`. Le nom n'a aucun effet sur le résultat.

Q13. Réécris cette boucle en compréhension de liste :

Réponse. `for o in ordres: prix.append(o["prix"])` devient :
`[ordre["prix"] for ordre in ordres]`

Q14. Comment ajouter un filtre à une compréhension de dictionnaire ou de set ?

Réponse. On ajoute une condition `if` à la fin, exactement comme pour une liste. Ex. set des symboles de vente : `{ordre["symbole"] for ordre in ordres if ordre["cote"] == "vente"}`.

Q15. Quelle est la différence entre une compréhension de liste et une expression génératrice ?

Réponse. La compréhension de liste calcule et stocke immédiatement tous les éléments en mémoire. L'expression génératrice produit les valeurs une à une, à la demande, sans tout stocker. Sur un flux de millions de ticks, le générateur évite de saturer la mémoire.

Q16. Quels délimiteurs produisent une liste, un dict, un set et un générateur ?

Réponse. Crochets [] pour une liste ; accolades avec deux-points { clé: valeur } pour un dict ; accolades simples { } pour un set ; parenthèses () pour un générateur.

Sur les f-strings et les chaînes

Q17. Comment afficher 26172.5 sous la forme « 26,172.50 \$ » ?

Réponse.

```
f"{26172.5: ,.2f} $" # -> "26,172.50 $"
```

La virgule demande le séparateur de milliers, .2f impose deux décimales.

Q18. À quoi servent respectivement split et join ?

Réponse. split découpe une chaîne en liste selon un séparateur ("ES,NQ".split(",") donne ["ES", "NQ"]). join fait l'inverse : il assemble une liste en une seule chaîne avec un séparateur choisi.

Q19. Dans " es ".strip().upper(), quel est l'ordre d'exécution et le résultat ?

Réponse. D'abord strip() retire les espaces ("es"), puis upper() met en majuscules ("ES"). Les méthodes s'enchaînent de gauche à droite, chacune travaillant sur le résultat de la précédente.

Sur les fonctions

Q20. Quelle est la différence entre un appel positionnel et un appel nommé ?

Réponse. En positionnel, c'est l'ordre des valeurs qui détermine le paramètre auquel chacune correspond. En nommé, on précise explicitement le nom de chaque paramètre, ce qui libère de l'ordre et améliore la lisibilité.

Q21. Quelle est la différence entre *args et **kwargs ?

Réponse. *args rassemble les arguments passés SANS nom (positionnels) dans un tuple. **kwargs rassemble les arguments passés AVEC un nom (nom=valeur) dans un dictionnaire. Ce qui décide n'est pas la donnée mais la façon de la passer : inspecter(5) met 5 dans args, inspecter(quantite=5) met 5 dans kwargs.

Q22. *args peut-il être une liste, ou **kwargs un set ?

Réponse. Non, jamais. *args est toujours un tuple et **kwargs toujours un dictionnaire. C'est imposé par Python, ce n'est pas un choix.

Q23. Explique le bug de def ajouter(ordre, panier=[]) et sa correction.

Réponse. La liste par défaut est créée une seule fois, à la définition de la fonction, puis partagée par tous les appels : les éléments s'y accumulent au lieu de repartir à vide. On corrige en mettant panier=None par défaut et en créant une liste neuve dans la fonction quand panier vaut None.

Sur les exceptions

Q24. Quelle clause s'exécute toujours, succès ou échec, et à quoi sert-elle ?

Réponse. La clause `finally`. Elle sert à libérer les ressources (fermer une connexion, relâcher un verrou, journaliser) quoi qu'il arrive.

Q25. Quel type d'erreur obtient-on en accédant à une clé absente d'un dict ? Et à un index hors limites d'une liste ?

Réponse. Une clé absente lève un `KeyError` ; un index hors limites lève un `IndexError`.

Q26. Pourquoi attraper `KeyError` et `ValueError` séparément plutôt qu'avec un seul `except` ?

Réponse. Pour réagir de façon adaptée à chaque cas : un `KeyError` révèle un ordre mal formé (champ manquant), un `ValueError` une règle métier non respectée. Des `except` distincts permettent un message et un traitement spécifiques.

Q27. Pourquoi placer `except Exception` en dernier ?

Réponse. Parce que les `except` sont évalués dans l'ordre, du plus spécifique au plus général. `except Exception` attrape tout (toutes les erreurs en héritent) ; placé en dernier, il sert de filet de sécurité pour l'imprévu sans masquer les cas précis traités au-dessus.

Q28. Comment crée-t-on sa propre exception, et avec quel mot-clé la déclenche-t-on ?

Réponse. On définit une classe qui hérite de `Exception` (`class OrdreInvalideError(Exception): pass`). On la déclenche avec le mot-clé `raise`, puis on peut l'attraper spécifiquement avec `except OrdreInvalideError`.

Q29. Que veut dire « fail gracefully » dans un système de trading ?

Réponse. Cela signifie qu'une erreur sur un ordre est attrapée et journalisée sans interrompre le traitement des autres ordres : le système reste en vie et continue de fonctionner plutôt que de s'arrêter brutalement.

Document suivant (2/8) : la programmation orientée objet — le dictionnaire « ordre » deviendra une classe `Ordre`, avec `dataclasses`, `enums` et méthodes spéciales.